

O que é uma Trigger?

Uma **Trigger (gatilho)** é um bloco de código SQL que é executado automaticamente quando ocorre um determinado evento em uma tabela.

Os eventos que podem disparar uma trigger são:

- **INSERT** → quando um registro é inserido.
- **UPDATE** → quando um registro é alterado.
- **DELETE** → quando um registro é removido.

Pense na trigger como um "vigia" da tabela:

"Sempre que alguém fizer determinada operação, execute automaticamente esta ação."

Exemplo do dia a dia

Imagine um sistema de vendas.

Temos a tabela: `produtos` com os campos: `id`, `nome`, `estoque`

E uma tabela: `pedidos`

Quando um pedido é inserido, queremos diminuir automaticamente o estoque do produto. Em vez de programar isso na aplicação, podemos criar uma Trigger.

Vantagens das Triggers

1. Automação

Executam tarefas automaticamente sem depender da aplicação.

Exemplo:

- Registrar logs
 - Atualizar estoque
 - Calcular totais
-

2. Integridade dos Dados

Garantem que determinadas regras sempre sejam executadas.

Exemplo:

- Impedir saldo negativo
 - Registrar alterações importantes
-

3. Centralização das Regras

A lógica fica no banco de dados.

Mesmo que:

- Java
- Node.js
- PHP
- Python

acessem o banco, a regra continuará funcionando.

4. Auditoria

Muito utilizadas para registrar:

- Quem alterou
 - Quando alterou
 - O que foi alterado
-

Tipos de Trigger no MySQL

BEFORE

Executa antes da operação.

```
BEFORE INSERT
BEFORE UPDATE
BEFORE DELETE
```

AFTER

Executa depois da operação.

```
AFTER INSERT
AFTER UPDATE
AFTER DELETE
```

Exemplo 1 - Trigger de Auditoria

Criando a tabela de clientes

```
CREATE TABLE clientes(
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100),
    email VARCHAR(100)
);
```

Criando a tabela de log

```
CREATE TABLE log_clientes(
    id INT AUTO_INCREMENT PRIMARY KEY,
    mensagem VARCHAR(255),
    data_evento DATETIME
);
```

Trigger

```
DELIMITER $$

CREATE TRIGGER trg_cliente_insert
AFTER INSERT ON clientes
FOR EACH ROW
BEGIN

    INSERT INTO log_clientes(
        mensagem,
        data_evento
    )
    VALUES(
        CONCAT('Cliente cadastrado: ', NEW.nome),
        NOW()
    );

END $$

DELIMITER ;
```

Testando

```
INSERT INTO clientes(nome,email)
VALUES('João','joao@email.com');
```

Resultado:

```
SELECT * FROM log_clientes;  
Cliente cadastrado: João foi inserido automaticamente.
```

Entendendo NEW

Dentro da Trigger existe:

```
NEW
```

que representa o registro novo.

Exemplo:

```
NEW.nome  
NEW.email
```

Entendendo OLD

Utilizado em UPDATE e DELETE.

Representa os dados antigos.

Exemplo:

```
OLD.nome  
OLD.email
```

Exemplo 2 - Histórico de Alterações

Tabela principal

```
CREATE TABLE produtos(  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(100),  
  preco DECIMAL(10,2)  
);
```

Tabela de histórico

```
CREATE TABLE historico_preco(  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  produto_id INT,  
  preco_antigo DECIMAL(10,2),  
  preco_novo DECIMAL(10,2),
```

```
        data_alteracao DATETIME
    );
```

Trigger

```
DELIMITER $$

CREATE TRIGGER trg_produto_update
AFTER UPDATE ON produtos
FOR EACH ROW
BEGIN

    INSERT INTO historico_preco(
        produto_id,
        preco_antigo,
        preco_novo,
        data_alteracao
    )
    VALUES (
        OLD.id,
        OLD.preco,
        NEW.preco,
        NOW()
    );

END $$

DELIMITER ;
```

Teste

```
UPDATE produtos
SET preco = 120.00
WHERE id = 1;
```

A alteração será gravada automaticamente no histórico.

Exemplo 3 - Controle de Estoque

Produtos

```
CREATE TABLE produtos(
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100),
    estoque INT
);
```

Pedidos

```
CREATE TABLE pedidos(  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  produto_id INT,  
  quantidade INT  
);
```

Trigger

```
DELIMITER $$  
  
CREATE TRIGGER trg_baixa_estoque  
AFTER INSERT ON pedidos  
FOR EACH ROW  
BEGIN  
  
  UPDATE produtos  
  SET estoque = estoque - NEW.quantidade  
  WHERE id = NEW.produto_id;  
  
END $$  
  
DELIMITER ;
```

Teste

Produto:

```
INSERT INTO produtos(nome,estoque)  
VALUES ('Notebook',10);
```

Pedido:

```
INSERT INTO pedidos(produto_id,quantidade)  
VALUES (1,2);
```

Estoque:

10 → 8 automaticamente.

Exemplo 4 - Validação de Dados

Suponha que não queremos permitir salário menor que R\$ 1.500.

Tabela

```
CREATE TABLE funcionarios(  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(100),  
  salario DECIMAL(10,2)  
);
```

Trigger

```
DELIMITER $$

CREATE TRIGGER trg_valida_salario
BEFORE INSERT ON funcionarios
FOR EACH ROW
BEGIN

    IF NEW.salario < 1500 THEN

        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salário abaixo do permitido';

    END IF;

END $$

DELIMITER ;
```

Teste

```
INSERT INTO funcionarios(nome,salario)
VALUES(
    'Maria',
    1200
);
```

Resultado:

```
ERROR: Salário abaixo do permitido
```

Cuidados ao utilizar Triggers

Embora sejam muito úteis, devem ser usadas com cautela:

Evite lógica complexa

Triggers muito grandes dificultam a manutenção.

Impacto na performance

Toda vez que ocorrer um INSERT, UPDATE ou DELETE, a trigger será executada.

Dificuldade de depuração

Às vezes a aplicação executa apenas:

```
INSERT INTO clientes ...
```

mas várias ações acontecem "por trás dos bastidores".

Resumo para iniciantes

Uma Trigger é um código SQL executado automaticamente quando ocorre um evento em uma tabela.

Evento	Quando executa
BEFORE INSERT	Antes de inserir
AFTER INSERT	Depois de inserir
BEFORE UPDATE	Antes de alterar
AFTER UPDATE	Depois de alterar
BEFORE DELETE	Antes de excluir
AFTER DELETE	Depois de excluir

Casos de uso mais comuns:

- Auditoria de alterações
- Histórico de dados
- Controle de estoque
- Validação de regras
- Atualização automática de informações
- Registro de logs

Para uma turma iniciante, o exemplo de **controle de estoque** e o de **auditoria de alterações** costumam ser os mais fáceis de entender, pois mostram claramente o benefício de automatizar regras diretamente no